

WEEK 5 – PAYMENT TRANSACTION MANAGEMENT SYSTEM

1. Scenario:

The Payment Transaction Management System is designed to maintain payment information associated with customer orders. The system records payment details, tracks transaction status, manages successful, failed, and pending payments, and generates payment-related business reports.

The Payment module is connected with the existing Customer and Order Management modules to provide a complete view of customer payment transactions.

2. Create Payment Table

The Payment table was created to store customer payment transaction details. The table contains important attributes such as Payment ID, Order ID, Payment Date, Payment Mode, Payment Status, and Transaction Amount.

The Payment ID uniquely identifies each payment, while the Order ID connects the payment transaction with the corresponding order. Payment Mode records the method used for the transaction, and Payment Status indicates whether the payment was successful, failed, or pending.

```
SQL> CREATE TABLE PAYMENT(Payment_ID VARCHAR(7),ORDER_ID VARCHAR(7),Payment_Date DATE,Payment_Mode VARCHAR(20),Payment_Status VARCHAR(10),Transaction_Amount NUMBER(10,2));
```

```
Table created.
```

3. Establishing Relationship with Orders

The Payment table was connected with the Orders table using the Order ID. This establishes a relationship between payment transactions and their corresponding customer orders.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT fk_Payment_Order FOREIGN KEY(Order_ID) REFERENCES Orders(Order_ID);  
Table altered.
```

4. Applying Data Validation Constraints

Data validation constraints were applied to the Payment table to maintain accuracy and consistency of the stored payment information.

The Transaction Amount was defined as a mandatory field so that every payment record contains an amount. A validation condition was also applied to ensure that the transaction amount is greater than zero.

```
SQL> ALTER TABLE Payment MODIFY(Transaction_Amount NOT NULL);  
Table altered.
```

```
SQL> ALTER TABLE Payment ADD CONSTRAINT chk_transaction_amount CHECK(Transaction_Amount>0);  
Table altered.
```

4.1 Payment Mode and Payment Status Validation

The Payment Mode was restricted to the available payment methods such as UPI, Card, Net Banking, and Cash. Similarly, Payment Status was restricted to Success, Failed, and Pending.

These constraints help prevent invalid values from being entered into the Payment table.

```
SQL> ALTER TABLE Payment ADD CONSTRAINT chk_payment_mode CHECK(Payment_Mode IN('UPI','Card','Net Banking','Cash'));
Table altered.

SQL> ALTER TABLE Payment ADD CONSTRAINT chk_payment_status CHECK(Payment_Status IN('Success','Failed','Pending'));
Table altered.
```

5. Store Payment Details

Payment transaction details were stored in the Payment table for different orders. The inserted records contain payment dates, payment modes, transaction amounts, and payment statuses.

The stored payment information forms the basis for further transaction and payment analysis.

```
SQL> INSERT INTO PAYMENT VALUES('PI01','2501',DATE '2025-08-02','UPI','Success',2500.00);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('PI02','2502',DATE '2025-09-12','Cash','Success',1405.00);
1 row created.
```

6. Display Payment Records

The stored payment records were displayed to verify the information entered into the Payment table. The displayed records contain different payment modes and transaction statuses.

```
SQL> SELECT * FROM Payment;
```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PI07	2507	18-SEP-26	UPI	Success	4200
PI08	2508	25-SEP-26	Cash	Pending	1850
PI09	2509	02-OCT-26	Card	Failed	6750
PI10	2510	05-OCT-26	Net Banking	Success	3200
PI01	2501	02-AUG-25	UPI	Success	2500
PI02	2502	12-SEP-25	Cash	Success	1405
PI03	2503	06-SEP-25	Card	Failed	25500
PI04	2504	22-SEP-25	UPI	Pending	5500
PI05	2505	22-OCT-25	Net Banking	Success	3600
PI06	2506	14-SEP-25	Cash	Failed	7650

```
10 rows selected.
```

7. Manage Successful, Failed and Pending Transactions

Payment transactions were analysed according to their payment status. The system identifies successful transactions, failed transactions, and transactions that are still pending.

This helps in monitoring the current status of customer payments and managing different types of payment transactions. The reference task specifically includes displaying successful payments, identifying failed transactions, and counting transactions by status.

7.1 Successful Payments

Successful payment transactions were displayed by filtering the Payment table based on the Success status.

7.2 Failed Payments

Failed payment transactions were identified separately to determine the transactions that were not successfully completed.

7.3 Pending Payments

Pending transactions were also identified to monitor payments that have not yet reached a successful or failed status.

```
SQL> SELECT * FROM Payment WHERE Payment_Status='Success';
```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PI07	2507	18-SEP-26	UPI	Success	4200
PI10	2510	05-OCT-26	Net Banking	Success	3200
PI01	2501	02-AUG-25	UPI	Success	2500
PI02	2502	12-SEP-25	Cash	Success	1405
PI05	2505	22-OCT-25	Net Banking	Success	3600

```
SQL> SELECT * FROM Payment WHERE Payment_Status='Failed';
```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PI09	2509	02-OCT-26	Card	Failed	6750
PI03	2503	06-SEP-25	Card	Failed	25500
PI06	2506	14-SEP-25	Cash	Failed	7650

```
SQL> SELECT * FROM Payment WHERE Payment_Status='Pending';
```

PAYMENT	ORDER_I	PAYMENT_D	PAYMENT_MODE	PAYMENT_ST	TRANSACTION_AMOUNT
PI08	2508	25-SEP-26	Cash	Pending	1850
PI04	2504	22-SEP-25	UPI	Pending	5500

8. Analyze Payment Methods Used by Customers

The payment methods used by customers were analysed to understand customer preferences and transaction distribution. The analysis includes the number of UPI transactions, card transactions, transaction count for each payment mode, and the most preferred payment method.

8.1 UPI Transactions

The number of transactions made using UPI was identified from the Payment table.

8.2 Card Payments

The number of card-based transactions was analysed from the available payment records.

8.3 Transaction Count by Payment Mode

The payment records were grouped according to Payment Mode to compare the number of transactions made through each method.

The analysis shows that **UPI and Cash have the highest transaction count in the available records**, followed by Card and Net Banking.

8.4 Most Preferred Payment Method

The payment method with the highest number of transactions was identified from the payment mode analysis.

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM PAYMENT WHERE PAYMENT_MODE = 'UPI';

UPI_TRANSACTIONS
-----
                3

SQL> SELECT COUNT(*) AS CARD_PAYMENTS FROM PAYMENT WHERE PAYMENT_MODE IN ('Credit Card','Debit Card');

CARD_PAYMENTS
-----
              0

SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;

PAYMENT_MODE      TRANSACTION_COUNT
-----
UPI                3
Cash               3
Card               2
Net Banking        2

SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC FETCH FIRST 1 ROW ONLY;

PAYMENT_MODE      TRANSACTION_COUNT
-----
UPI                3
```

9. Top Two Payment Methods

The payment modes were arranged according to their transaction count to identify the two most frequently used payment methods.

The result shows **UPI and Cash** as the top two payment methods, with three transactions each.

```
SQL> SELECT *
  2 FROM (
  3     SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT
  4     FROM PAYMENT
  5     GROUP BY PAYMENT_MODE
  6     ORDER BY TRANSACTION_COUNT DESC
  7 )
  8 WHERE ROWNUM <= 2;
```

PAYMENT_MODE	TRANSACTION_COUNT
UPI	3
Cash	3

10. Revenue Analysis

Revenue analysis was performed using successful payment transactions. The analysis helps determine the revenue generated through different payment methods.

The revenue was grouped according to Payment Mode to compare the contribution of each successful payment method. The average transaction amount was also calculated. These are the revenue-analysis requirements specified in the Week 5 reference.

```
SQL> SELECT PAYMENT_MODE, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM PAYMENT WHERE PAYMENT_STATUS = 'Success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;
```

PAYMENT_MODE	TOTAL_REVENUE
Net Banking	6800
UPI	6700
Cash	1405

11. Average Transaction Amount

The average transaction amount for successful payments was calculated to determine the average value of completed payment transactions.

The calculated average successful transaction amount is **₹2,981** based on the available payment records.

```
SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT),2) AS AVERAGE_TRANSACTION_AMOUNT FROM PAYMENT WHERE PAYMENT_STATUS = 'Success';
```

AVERAGE_TRANSACTION_AMOUNT
2981

12. Customer Payment History

Customer payment history was generated by combining information from the Customer, Orders, and Payment tables.

The resulting report displays customer name, order ID, order date, payment date, payment mode, transaction amount, and payment status. This provides a complete view of the payment activity associated with each customer order. The reference specifically requires customer name, Order ID, payment mode, amount, and payment status.

```
SQL> SELECT C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME, O.ORDER_ID AS ORDER_ID, O.ORDER_DATE AS ORDER_DATE, P.PAYMENT_DATE AS PAYMENT_DATE, P.PAYMENT_MODE AS PAYMENT_MODE, P.TRANSACTION_AMOUNT AS AMOUNT, P.PAYMENT_STATUS AS PAYMENT_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN PAYMENT P ON O.ORDER_ID = P.ORDER_ID ORDER BY P.PAYMENT_DATE;
```

CUSTOMER_NAME		ORDER_ID	ORDER_DAT	PAYMENT_D
Lira James		2501	02-AUG-26	02-AUG-25
UPI	2500 Success			
Kairo William		2503	06-SEP-26	06-SEP-25
Card	25500 Failed			
Kenny Robert		2502	12-SEP-26	12-SEP-25
Cash	1405 Success			

CUSTOMER_NAME		ORDER_ID	ORDER_DAT	PAYMENT_D
Lira James		2506	14-SEP-26	14-SEP-25
Cash	7650 Failed			
Suxi Martin		2504	22-SEP-26	22-SEP-25
UPI	5500 Pending			

13. Payment Analysis

A combined payment analysis was performed to compare different payment methods based on transaction count, total revenue, and average transaction amount.

This analysis provides a consolidated view of payment performance and helps understand how different payment methods contribute to the overall transaction activity and revenue.

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT, SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE, ROUND(AVG(TRANSACTION_AMOUNT), 2) AS AVG_TRANSACTION FROM PAYMENT WHERE PAYMENT_STATUS = 'Success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;
```

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
Net Banking	2	6800	3400
UPI	2	6700	3350
Cash	1	1405	1405